

5-dars

Phase 4: Deployment & Inference umumiy mazmuni

Image Classification modelini yaratishda bir nechta katta bosqich bo‘ladi. Avval data tayyorlanadi, keyin model architecture quriladi, undan keyin model training va optimization orqali o‘qitiladi. Training tugagandan keyin model hali to‘liq product hisoblanmaydi. Model haqiqiy foydalanuvchi ishlatadigan tizimga joylashtirilgandan keyingina amaliy foyda bera boshlaydi. Shu bosqich Deployment & Inference deb ataladi.

Phase — bosqich degani. Phase 4 — to‘rtinchi bosqich. Bu bosqich modelni real hayotda ishlatishga tayyorlash, yangi rasm kelganda prediction olish, modelning ishlashini kuzatish va foydalanuvchiga final natija qaytarish bilan bog‘liq.

Deployment — train qilingan modelni real tizimga joylash va ishga tushirish jarayoni. Train qilingan model notebook ichida yoki lokal papkada qolib ketishi mumkin. Bunday holatda undan oddiy foydalanuvchi foydalana olmaydi. Deployment orqali model web app, API, server, mobile app yoki edge device ichida ishlaydigan holatga keltiriladi.

Inference — training tugagandan keyin modelning yangi data ustida javob berish jarayoni. Yangi rasm keladi, rasm preprocessing qilinadi, model forward pass bajaradi va class probabilitylarni chiqaradi.

Inference paytida weightlar o‘zgarmaydi. Model faqat oldin o‘rgangan bilimidan foydalanib prediction beradi.

Monitoring — deploy qilingan modelning vaqt o‘tishi bilan qanday ishlayotganini kuzatish. Model real tizimga chiqqandan keyin ham uning tezligi, aniqligi, xatolari, data drift va foydalanuvchi requestlari

kuzatib boriladi. Monitoring bo‘lmasa, model sekinlashgani yoki noto‘g‘ri natija bera boshlagani vaqtida bilinmay qolishi mumkin.

Optimization — modelni production uchun tezroq, yengilroq va qulayroq qilish jarayoni. Production — haqiqiy foydalanuvchi ishlatadigan real tizim muhiti. Model training vaqtida katta bo‘lishi mumkin, lekin productda tez ishlashi kerak. Shu sababli quantization, ONNX export, TensorRT kabi usullar ishlatiladi.

Final Model — training, optimization va deploymentdan o‘tgan, foydalanuvchiga prediction qaytara oladigan tayyor model. Final model yangi rasmga predicted class va probability score qaytaradi. Predicted class — model tanlagan class. Probability score — modelning ishonch darajasi.

Phase 4 umumiy pipeline quyidagicha tushuniladi:

Deployment Optimization

↓

Inference

↓

Monitoring

↓

Final Output

Deployment Optimization qismida model yengillashtiriladi va tezlashtiriladi. Inference qismida yangi rasmga prediction olinadi. Monitoring qismida modelning real ishlashi kuzatiladi. Final Output qismida foydalanuvchiga class va probability natijasi qaytariladi.

Chap tomonda vertikal oqim sifatida Phase 4 ichidagi ketma-ketlik ko‘rsatilgan: deployment optimization, inference, monitoring va final model. Bu oqim model trainingdan keyingi real hayot bosqichi bitta qadam emasligini, bir nechta alohida vazifadan iboratligini ko‘rsatadi.

Har bir qism mustaqil vazifa bajaradi, lekin hammasi birga ishlaganda model product darajasiga chiqadi.

Deployment Optimization

Deployment Optimization — train qilingan modelni production uchun yengillashtirish, tezlashtirish va ko‘chirishga qulay holatga keltirish jarayoni. Model trainingda yaxshi accuracy bergani bilan productda og‘ir, sekin yoki katta hajmli bo‘lsa, foydalanuvchi tajribasi yomonlashadi. Shu sababli modelni deploymentdan oldin optimization qilish kerak bo‘ladi.

Deployment Optimization quyidagi maqsadlarga xizmat qiladi:

- model hajmini kamaytirish
- inference tezligini oshirish
- latency qiymatini kamaytirish
- turli muhitlarda ishlatishga qulay qilish
- server yoki qurilma resursini tejash

Model Size — model faylining hajmi. Masalan, model 500 MB bo‘lsa, serverda saqlash, yuklash va ishga tushirish og‘ir bo‘lishi mumkin. Optimizationdan keyin model kichrayishi mumkin.

Inference Speed — model yangi rasmga qanchalik tez prediction berishi. Speed — tezlik. Inference speed yuqori bo‘lsa, foydalanuvchi natijani tezroq oladi.

Latency — bitta request kelgandan javob qaytguncha ketgan vaqt. Masalan, foydalanuvchi rasm yukladi va model 120 millisekundda javob berdi. Shu 120 ms latency hisoblanadi. Millisecond yoki ms — sekundning mingdan bir qismi.

Portability — modelni turli muhitlarda ishlatish qulayligi. Masalan, model PyTorchda train qilingan bo'lsa, ONNX format orqali boshqa runtimelarda ham ishlashi mumkin. Runtime — model ishga tushadigan bajarilish muhiti.

Deployment Optimizationni oddiy hayotiy misol bilan tushunish mumkin. Katta faylni telefonga yuborishdan oldin zip qilinsa, fayl ichidagi asosiy ma'lumot o'zgarmaydi, lekin uni uzatish yoki saqlash osonlashadi. Model optimization ham shunga o'xshaydi: modelning maqsadi o'zgarmaydi, lekin u yengilroq, tezroq va product uchun qulayroq bo'ladi.

Optimization usullari ichida uchta asosiy tushuncha ko'p ishlatiladi:

Quantization

ONNX

TensorRT

Quantization

Quantization — model ichidagi son formatlarini kichraytirish orqali model hajmini kamaytirish va inference tezligini oshirish usuli. Deep Learning modelida weightlar ko'pincha float32 formatda saqlanadi. Float32 — 32 bitli kasr son formati. Bu format aniq, lekin hajmi katta va hisoblashda og'irroq bo'lishi mumkin.

Quantization orqali float32 qiymatlar int8 kabi kichik formatga o'tkazilishi mumkin. Int8 — 8 bitli butun son formati. Bu modelni ancha kichraytiradi va ayrim qurilmalarda inference tezligini oshiradi.

Oddiy ko'rinish:

Float32 → Int8

Bu jarayonda model ancha yengillashadi. Ba'zan accuracy ozgina kamayishi mumkin, lekin tezlik va hajm bo'yicha katta foyda beradi. Productda ayniqsa mobile app, edge device va real-time tizimlar uchun quantization muhim bo'ladi.

ONNX

ONNX — Open Neural Network Exchange. O'zbekcha izohi: neural network modellarini turli framework va runtime orasida ko'chirishga yordam beradigan universal format.

Framework — model yaratish va train qilish uchun ishlatiladigan dasturiy vositalar to'plami. Masalan, PyTorch va TensorFlow framework hisoblanadi.

PyTorchda train qilingan modelni to'g'ridan-to'g'ri hamma joyda ishlatish qulay bo'lmasligi mumkin. ONNX formatga export qilinganda model boshqa runtime yoki deployment muhitida ishlashi osonlashadi.

Export — modelni boshqa formatga chiqarish.

ONNX quyidagi holatlarda foydali:

PyTorch modelni boshqa muhitda ishlatish
modelni ONNX Runtime orqali tezroq inference qilish
server yoki edge device uchun universal formatga
o'tkazish
turli frameworklar orasida moslashuvni oshirish

ONNX model portabilityni oshiradi. Portability — modelni boshqa joyga ko'chirish va ishlatish qulayligi.

TensorRT

TensorRT — NVIDIA GPU uchun optimallashtirilgan inference engine. Engine — bajaruvchi mexanizm yoki ishga tushirish tizimi. TensorRT modelni NVIDIA GPUda tezroq ishlashga moslashtiradi.

TensorRT ayniqsa real-time inference uchun foydali. Real-time — real vaqt rejimida ishlash. Masalan, kamera videosini tez tahlil qilish, har bir framega tez javob berish yoki serverda ko‘p so‘rovga javob qaytarish kerak bo‘lsa, TensorRT yordam beradi.

TensorRT quyidagilarni yaxshilashi mumkin:

lower latency

higher throughput

smaller size

optimized runtime

Lower latency — javob kechikish vaqtini kamaytirish.

Higher throughput — bir vaqt ichida ko‘proq so‘rovni qayta ishlash.

Smaller size — model hajmini kamaytirish.

Optimized runtime — tezlashtirilgan bajarilish muhiti.

Foizli ko‘rsatkichlar misol sifatida olinsa, optimizationdan keyin model size 60% darajada kichrayishi, inference speed 100% yaxshilanishi, latency reduction 88% bo‘lishi, portability 78% yaxshilanishi ko‘rsatiladi. Bu raqamlar optimization natijasida modelning bir nechta tomondan foydaliroq bo‘lishini tushuntiradi. Model faqat accuracy bilan baholanmaydi; product uchun hajm, tezlik, latency va portability ham juda muhim.

Deployment nima?

Deployment — train qilingan modelni real foydalanuvchi ishlata oladigan tizimga joylash va ishga tushirish jarayoni. Model faqat notebook ichida ishlasa, u hali product emas. Foydalanuvchi rasm yuklab natija olishi uchun model API, web app, mobile app, server yoki edge device ichida ishlashi kerak.

Deployment hayotiy o‘xshatmada taom tayyorlashga o‘xshaydi. Taom tayyor bo‘lishi trainingga o‘xshaydi. Uni chiroyli tarelkaga joylab mijozga berish esa deploymentga o‘xshaydi. Model trainingdan o‘tgan bo‘lishi mumkin, lekin foydalanuvchi ishlata olmasa, amaliy qiymati kam bo‘ladi.

Deployment flow quyidagicha:

Trained Model

↓

Optimize

↓

Wrap in API/App

↓

Deploy to Server/Device

↓

User Uses It

Trained Model — o‘qitilgan model. Masalan, `best_model.pth` faylida saqlangan PyTorch modeli. `.pth` — PyTorch model weightlari saqlanadigan fayl formati.

Optimize — modelni tezroq va yengilroq qilish. Bu qismda quantization, ONNX yoki TensorRT ishlatilishi mumkin.

Wrap in API/App — modelni API yoki app ichiga o‘rash. Wrap — o‘rash yoki modelni boshqa tizim ichida ishlaydigan ko‘rinishga keltirish. API — Application Programming Interface. API boshqa dasturlar modelga so‘rov yuborishi va javob olishi uchun xizmat qiladi. App — application, ya’ni ilova.

Deploy to Server/Device — modelni server yoki qurilmaga joylash. Server — foydalanuvchilardan request qabul qiladigan va javob qaytaradigan kompyuter yoki cloud muhit. Device — qurilma, masalan telefon, laptop, Raspberry Pi, Jetson yoki kamera.

User Uses It — foydalanuvchi modeldan foydalanadi. Masalan, foydalanuvchi rasm yuklaydi va model natija qaytaradi.

Deployment nima uchun kerak deganda, asosiy sabab model notebook ichida qolib ketmasligi va real foydalanuvchi foydalana olishi hisoblanadi. Model local PC, server, web app, mobile app yoki edge devicega deploy qilinishi mumkin. Deployment natijasida foydalanuvchi rasm yuklaydi va predicted class hamda probability natijasini oladi.

Deployment oddiy model faylini ishga tushirish emas. Unda quyidagilar ham hisobga olinadi:

- model faylini yuklash
- preprocessingni bir xil qilish
- API yoki UI yaratish
- serverda ishlatish
- xatolarni boshqarish
- monitoring qilish
- security va scalabilityni o‘ylash

UI — User Interface, foydalanuvchi ko‘radigan interfeys. Masalan, rasm yuklash tugmasi va natija oynasi.

Security — xavfsizlik. Modelga noto‘g‘ri, zararli yoki juda katta fayl yuborilganda tizim buzilmasligi kerak.

Scalability — tizim ko‘p foydalanuvchiga moslasha olishi. Agar 10 ta emas, 1000 ta foydalanuvchi rasm yuborsa, server buni ko‘tara olishi kerak.

Deployment & Inference umumiy texnik oqimi

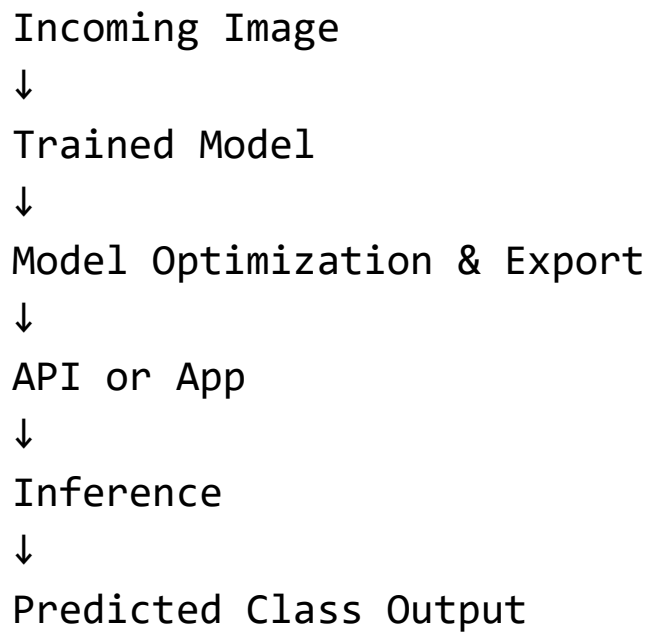
Deployment & Inference jarayoni sodda va texnik ko‘rinishda tushuntiriladi.

Sodda ko‘rinishda foydalanuvchi rasm yuklaydi. Rasm app yoki servicega boradi. Service — xizmat. App yoki service ichida trained model ishlaydi. Model rasmni tahlil qiladi va prediction qaytaradi. Masalan:

```
User uploads image
↓
App / Service
↓
Trained Model
↓
Prediction
↓
Returns prediction: Dog 98%
```

Bu oqimda foydalanuvchi texnik tafsilotlarni ko‘rmaydi. U faqat rasm yuklaydi va natija oladi. Masalan, rasmda it bo‘lsa, tizim **Dog 98%** natija qaytaradi. Bu model rasmni 98% ishonch bilan dog deb topganini bildiradi.

Texnik ko‘rinishda jarayon ko‘proq qismlardan iborat:



Incoming Image — foydalanuvchi yuklagan yangi rasm.

Trained Model — trainingdan o‘tgan model. Bu model deploymentga tayyor bo‘ladi.

Model Optimization & Export — modelni tezroq va yengilroq qilish hamda boshqa formatga chiqarish. Bu qismda quantization, ONNX va TensorRT ishlatilishi mumkin.

API or App — model API yoki app ichida ishlaydi. Deployed service — deploy qilingan xizmat, ya’ni foydalanuvchi ishlata oladigan tizim.

Inference — kelgan rasm model orqali qayta ishlanadi va prediction hosil qilinadi.

Predicted Class Output — model yakuniy class va probability score qaytaradi. Masalan:

Dog: 98%

Bu jarayon shuni ko‘rsatadi: deployment faqat modelni serverga qo‘yish emas, balki modelni optimization qilish, APIga ulash,

incoming image'ni preprocessing qilish, inference bajarish va output qaytarishni ham o'z ichiga oladi.

Inference nima?

Inference — training tugagandan keyin modelning yangi data ustida prediction berish jarayoni. Trainingda model o'rganadi, inference'da esa model javob beradi. Inference paytida weightlar yangilanmaydi. Backpropagation ishlamaydi. Faqat forward pass ishlaydi.

Forward Pass — input modeldan oldinga qarab o'tib output hosil qiladigan jarayon. Inference'da model faqat forward pass orqali prediction chiqaradi.

Inference jarayoni odatda quyidagi ketma-ketlikda bo'ladi:

1. Input Image
2. Preprocess
3. Model Runs
4. Probabilities
5. Predicted Class

Input Image — foydalanuvchi yuborgan rasm. Masalan, mushuk yoki it rasmi.

Preprocess — rasmni modelga mos holatga keltirish. Training vaqtida qanday preprocessing qilingan bo'lsa, inference vaqtida ham shunga yaqin preprocessing bajarilishi kerak. Aks holda model noto'g'ri natija berishi mumkin.

Preprocessing ichiga quyidagilar kiradi:

resize
normalize
center crop

tensor formatga o'tkazish
batch dimension qo'shish

Resize — rasm o'lchamini o'zgartirish. Masalan, 224×224 formatga keltirish.

Normalize — pixel qiymatlarini ma'lum oraliqqa keltirish.

Center crop — rasm markaziy qismini kesib olish.

Tensor — ko'p o'lchamli raqamlar massivi. Model rasmni tensor sifatida qabul qiladi.

Batch dimension — model bitta rasmni ham batch ko'rinishida kutishi mumkin. Masalan, bitta rasm $[3, 224, 224]$ bo'lsa, batch dimension qo'shilgandan keyin $[1, 3, 224, 224]$ bo'ladi. Bu yerda 1 bitta rasm borligini bildiradi.

Model Runs — model rasmni qayta ishlaydi. Bu bosqichda trained model forward pass bajaradi.

Probabilities — model classlar bo'yicha ehtimolliklar chiqaradi. Masalan:

Cat: 0.02

Dog: 0.98

Predicted Class — eng yuqori probabilityga ega class tanlanadi. Yuqoridagi misolda predicted class Dog bo'ladi.

Training mode va inference mode o'rtasidagi asosiy farq mavjud.

Training mode'da weightlar o'zgaradi, backpropagation bor.

Inference mode'da weightlar o'zgarmaydi, faqat forward pass ishlaydi.

Training mode — model o‘rganadigan holat. Dropout va batch normalization kabi qatlamlar training rejimida boshqacha ishlashi mumkin.

Inference mode — model javob beradigan holat. PyTorchda inference’dan oldin `model.eval()` ishlatiladi. `eval()` — evaluation mode, ya’ni modelni prediction holatiga o‘tkazish.

Natija probability scores va predicted class ko‘rinishida chiqadi. Probability scores — har bir class uchun ehtimollik qiymatlari.

Inference simple view va technical view

Inference simple view ko‘rinishida foydalanuvchi rasm yuklaydi. Model rasmni tahlil qiladi va predicted class qaytaradi.

Misol sifatida foydalanuvchi laptop orqali rasm yuklaydi. Rasm trained modelga boradi. Model natija sifatida `Cat` classini chiqaradi va confidence 98% bo‘ladi.

Confidence — modelning ishonch darajasi. `Cat confidence 98%` degani model rasmni 98% ishonch bilan cat deb topganini bildiradi.

Simple view oqimi:

User uploads image

↓

Trained model

↓

Predicted class

↓

Cat, confidence 98%

Technical view esa inference ichidagi qadamlarni aniqroq ko‘rsatadi:

1. Preprocess
2. `model.eval()`
3. Forward Pass
4. Probabilities
5. Predicted Class

Preprocess qismida `resize`, `normalize` va `center crop` bajariladi. `Resize` rasm o'lchamini modelga moslaydi. `Normalize` pixel qiymatlarini trainingdagi scale bilan mos qiladi. `Center crop` rasmning markaziy qismini oladi.

`model.eval()` modelni evaluation modega o'tkazadi. Bu inference paytida muhim, chunki `dropout` va `batch normalization` kabi qatlamlar `prediction` rejimida to'g'ri ishlashi kerak.

Forward Pass qismida `input image` model orqali o'tadi. Bu yerda `weight update` bo'lmaydi.

Probabilities qismida model classlar bo'yicha scorelarni probabilityga aylantiradi. Misol sifatida quyidagi natija olinadi:

Cat: 0.98
Dog: 0.02
Bird: 0.00
Car: 0.00

Predicted Class qismida eng katta probability tanlanadi. Bu holatda Cat classi tanlanadi. Confidence 98%.

Bu oqim inference'da training loop ishlamasligini ko'rsatadi. Inference uchun `loss`, `backward pass`, `optimizer step` kerak emas. Faqat preprocess, model forward pass, softmax va class tanlash kerak bo'ladi.

Deployment vs Inference farqi

Deployment va Inference bir-biriga bog‘liq, lekin ular bir xil narsa emas.

Deployment — modelni real tizimga joylashtirish bosqichi. Bu system-level task hisoblanadi. System-level task — butun tizim darajasidagi vazifa. Deploymentda server, API, app, edge device, optimization, packaging, scalability va security haqida o‘ylash kerak.

Packaging — model va kodni ishlatishga tayyor paket ko‘rinishiga keltirish.

Scalability — tizim ko‘p foydalanuvchini ko‘tara olishi.

Security — xavfsizlik.

Inference — modelning yangi data ustida prediction berishi. Bu prediction-level task hisoblanadi. Prediction-level task — bitta prediction olish darajasidagi vazifa. Inference har bir user request kelganda bajarilishi mumkin.

Deployment haqida asosiy fikrlar:

model real tizimga joylashtiriladi
server, API, app yoki edge devicega chiqariladi
optimization, packaging, scalability, security
o‘ylanadi
maqsad: foydalanuvchi modeldan foydalana olishi
bir martalik yoki kam-kam takrorlanadigan setup
jarayoni
system-level task

Inference haqida asosiy fikrlar:

yangi dataga model javob beradi
forward pass ishlaydi
weight update bo'lmaydi
har bir user requestda ishlashi mumkin
maqsad: prediction olish
doimiy ravishda qayta-qayta bajariladi
prediction-level task

Deployment misoli: model FastAPI serveriga joylanadi va internet orqali rasm qabul qila oladigan holatga keltiriladi.

Inference misoli: foydalanuvchi rasm yuboradi, model rasmni Dog 0.98 deb topadi.

Oddiy farq:

Deployment = modelni qayerda va qanday ishlatishga tayyorlash

Inference = modelga yangi rasm berilganda javob olish

Deployment bo'lmasa, oddiy foydalanuvchi modeldan foydalana olmaydi. Inference bo'lmasa, deploy qilingan model foydalanuvchiga prediction bera olmaydi. Shuning uchun ikkalasi birga kerak bo'ladi.

Monitoring nima?

Monitoring — deploy qilingan modelning vaqt o'tishi bilan ishlashini kuzatish jarayoni. Model productga chiqqandan keyin uni tashlab qo'yish to'g'ri emas. Real hayotdagi data o'zgarishi, server sekinlashishi, foydalanuvchilar boshqacha rasm yuborishi yoki model accuracy pasayishi mumkin. Monitoring shu muammolarni vaqtida ko'rishga yordam beradi.

Monitoring hayotiy misolda mashina panelini kuzatishga o'xshaydi. Mashina haydalayotganda spidometr, benzin, harorat va ogohlantirish chiroqlari kuzatiladi. Agar benzin kamaysa yoki harorat oshsa, haydovchi vaqtida chora ko'radi. Model monitoring ham shunday ishlaydi: model tezligi, accuracy, xatolar va drift kuzatiladi.

Monitoringda asosiy kuzatiladigan tushunchalar:

Accuracy

Latency

Drift

Failures

Accuracy

Accuracy — model predictionlarining to'g'riligini kuzatadi. Agar model cat vs dog classification qilsa, haqiqiy cat va dog rasmlarida nechta to'g'ri javob bergani accuracy orqali baholanadi.

Accuracy pasayishi model real data'da yomonroq ishlay boshlaganini bildirishi mumkin.

Latency

Latency — bitta request qancha vaqtda javob qaytarishini o'lchaydi.

Request — foydalanuvchi yuborgan so'rov. Masalan, foydalanuvchi rasm yukladi va model 120 ms ichida javob berdi. Shu 120 ms latency hisoblanadi.

Latency yuqori bo'lsa, foydalanuvchi uzoq kutadi. Productda bu user experience yomonlashishiga olib keladi.

User experience — foydalanuvchining tizimdan foydalanish tajribasi.

Drift

Drift — real data training data'dan uzoqlashib ketishi. Model train qilingan data ma'lum ko'rinishda bo'ladi. Real hayotda kelayotgan rasmlar esa boshqa yorug'likda, boshqa kamera bilan, boshqa fon yoki boshqa sifatda bo'lishi mumkin. Bu data drift deyiladi.

Data drift bo'lsa, model accuracy pasayishi mumkin. Masalan, model faqat yorug' joydagi it rasmlarida train qilingan, lekin real foydalanuvchilar qorong'i joydagi it rasmlarini yuboradi. Model xato qilishni boshlashi mumkin.

Failures

Failures — xatolar, crash va timeoutlarni kuzatish. Crash — dastur to'xtab qolishi. Timeout — server belgilangan vaqt ichida javob qaytara olmasligi.

Failures monitoring qilinmasa, foydalanuvchilar xato olishi mumkin, lekin tizim egasi muammoni vaqtida bilmasligi mumkin.

Monitoring kerak bo'lishining asosiy sababi shuki, model sekinlashgani, noto'g'ri class berayotgani yoki data drift paydo bo'lgani vaqtida aniqlanadi. Monitoring bo'lmasa, model productda ishlayapti deb o'ylanadi, lekin aslida foydalanuvchilarga yomon natija qaytarayotgan bo'lishi mumkin.

Monitoring simple view va technical view

Monitoring simple view ko'rinishida live monitoring dashboard orqali tushuntiriladi. Dashboard — kuzatuv paneli. Unda speed va accuracy kabi ko'rsatkichlar ko'rinadi.

Monitoring dashboard misolida speed yoki latency 120 ms, accuracy 98.2% ko'rsatiladi. 120 ms yaxshi latency sifatida baholanadi. Accuracy 98.2% ham yaxshi natija hisoblanadi. Yon tomonda

performance real time kuzatiladi, muammolar erta aniqlanadi va model sogʻlom holatda saqlanadi degan mazmun bor.

Real time — ayni vaqtda, jonli kuzatish.

Technical viewda monitoring bir nechta alohida koʻrsatkich orqali qilinadi:

Latency Response Time

Accuracy

Data Drift

Prediction Errors

Latency Response Time misolida 120 ms qiymat koʻrsatiladi. Shuningdek, P50, P95 va P99 kabi qiymatlar ham ishlatiladi.

P50 — requestlarning 50 foizi shu vaqt ichida javob olganini bildiradi.

P95 — requestlarning 95 foizi shu vaqt ichida javob olganini bildiradi.

P99 — requestlarning 99 foizi shu vaqt ichida javob olganini bildiradi.

Masalan:

P50 = 110 ms

P95 = 220 ms

P99 = 350 ms

Bu tizimning oddiy va ogʻirroq holatlardagi javob tezligini koʻrsatadi. P99 katta boʻlsa, ayrim foydalanuvchilar juda uzoq kutayotgan boʻlishi mumkin.

Accuracy misolida 98.2% qiymat ko'rsatiladi va target 95% dan yuqori bo'lishi kerakligi ko'rsatiladi. Target — kutilgan minimal maqsad. Accuracy targetdan yuqori bo'lsa, holat yaxshi.

Data Drift misolida drift qiymati 0.18 ko'rsatiladi. PSI kabi drift metric ishlatilishi mumkin. PSI — Population Stability Index. Bu training data va real data taqsimoti orasidagi farqni o'lchashda ishlatiladi. Warning threshold 0.2 bo'lishi mumkin. Threshold — chegara qiymat. 0.18 qiymat 0.2 ga yaqin bo'lgani uchun warning holatiga yaqin deb qaraladi.

Warning — ogohlantirish.

Prediction Errors misolida xatolar 0.7% ko'rsatiladi. Target 1% dan kichik bo'lishi kerak. 0.7% targetdan past bo'lgani uchun yaxshi holatga yaqin. Agar prediction errors 1% dan oshsa, alert berilishi mumkin.

Alert — jiddiy ogohlantirish.

Monitoring statuslari uch xil bo'lishi mumkin:

Good

Warning

Alert

Good — yaxshi holat.

Warning — ogohlantirish, ehtiyot bo'lish kerak.

Alert — muammo jiddiy, tekshirish kerak.

Monitoring real productda modelni sog'lom saqlash, xatolarni erta ko'rish va kerak bo'lsa retraining qilish uchun zarur.

Retraining — modelni yangi data bilan qayta train qilish.

Cat vs Dog Classifier real ishlatish misoli

Cat vs Dog image classification modeli real web appga deploy qilinadi. Bu misol end-to-end product oqimini ko'rsatadi. End-to-end — boshidan oxirigacha to'liq jarayon.

Scenario — vaziyat. Cat vs dog model bor, u web appga joylanadi. Foydalanuvchi rasm yuklaydi, app rasmni preprocessing qiladi, model inference bajaradi, output sifatida Dog 0.98 qaytaradi va natija log hamda monitoring tizimiga yoziladi.

Oqim quyidagicha:

1. User uploads image
2. App preprocesses image
3. Model inference
4. Output: Dog 0.98
5. Log & monitor

User uploads image — foydalanuvchi rasm yuklaydi.

App preprocesses image — ilova rasmni modelga mos holatga keltiradi. Masalan, resize, normalize, tensor format.

Model inference — model yangi rasmga prediction beradi.

Output: Dog 0.98 — model natijasi. Dog classi 0.98 probability bilan tanlangan.

Log & monitor — natija, response time va prediction distribution yozib boriladi. Log — tizimda sodir bo'lgan voqealar yozuvi.

Prediction distribution — model qaysi classlarni qanchalik ko'p prediction qilayotganini ko'rsatadigan taqsimot.

Project ichidagi qismlar:

Frontend
Backend
Model Runtime
Monitoring

Frontend — foydalanuvchi ko‘radigan qism. Masalan, rasm yuklash tugmasi, preview va natija oynasi. Gradio, Streamlit yoki Web frontend bo‘lishi mumkin.

Gradio — Machine Learning modellari uchun tez UI yaratishga qulay Python vositasi.

Streamlit — Python orqali web app yaratishga qulay vosita.

Backend — requestni qabul qiladigan va modelga yuboradigan server qismi. FastAPI yoki Flask backend uchun ishlatilishi mumkin.

FastAPI — Python’da tez va zamonaviy API yaratish frameworki.

Flask — yengil Python web framework.

Model Runtime — model ishlaydigan muhit. PyTorch, ONNX Runtime yoki TensorRT bo‘lishi mumkin.

PyTorch — Deep Learning framework.

ONNX Runtime — ONNX modelni tez ishga tushirish uchun runtime.

TensorRT — NVIDIA GPU uchun tezlashtirilgan inference engine.

Monitoring — logs, response time va prediction distribution kuzatiladi. Response time — javob qaytarish vaqti. Prediction distribution orqali model hamma rasmni bitta classga chiqarib yuboryaptimi yoki classlar bo‘yicha tabiiy taqsimlanyaptimi ko‘rish mumkin.

Bu misol real product uchun eng sodda architecturelardan biri hisoblanadi. Foydalanuvchi frontend orqali rasm yuklaydi, backend rasmni qabul qiladi, model runtime prediction qiladi, backend natijani frontendga qaytaradi, monitoring tizimi esa jarayonni kuzatadi.

Simple Deployment Flow: PyTorch → FastAPI / Gradio

Simple Deployment Flow train qilingan PyTorch modelni inference app yoki API orqali ishlatishga olib chiqish bosqichlarini ko'rsatadi. Bu jarayonda training loop kerak emas. Asosiy vazifa modelni yuklash, eval modega o'tkazish, rasmni preprocessing qilish, inference bajarish va natijani foydalanuvchiga ko'rsatishdan iborat.

Amaliy bosqichlar:

1. Modelni load qilish
2. `model.eval()` qilish
3. Input image ni preprocess qilish
4. `with torch.no_grad()` bilan inference
5. Softmax va predicted class chiqarish
6. UI yoki API orqali foydalanuvchiga natijani ko'rsatish

Modelni load qilish — saqlangan model weightlarini yuklash.

Masalan, `best_model.pth` fayli ishlatiladi.

```
model.load_state_dict(torch.load("best_model.pth"))
```

`load_state_dict` — model weightlarini model architecture ichiga yuklaydi.

`torch.load` — PyTorch faylni o'qish funksiyasi.

`best_model.pth` — training davomida eng yaxshi natija bergan model weightlari saqlangan fayl.

`model.eval()` — modelni evaluation yoki inference modega o'tkazadi. Bu inference uchun juda muhim.

`model.eval()`

`with torch.no_grad()` — gradient hisoblashni o'chiradi. Inference paytida gradient kerak emas, chunki weightlar yangilanmaydi. Bu xotirani tejaydi va inference'ni tezlashtiradi.

```
with torch.no_grad():  
    image = preprocess(img).unsqueeze(0)  
    outputs = model(image)  
    probs = torch.softmax(outputs, dim=1)  
    pred = probs.argmax(dim=1).item()
```

`preprocess(img)` — rasmni modelga mos formatga keltiradi. Resize, normalization va tensor conversion bo'lishi mumkin.

`unsqueeze(0)` — batch dimension qo'shadi. Bitta rasm modelga batch ko'rinishida beriladi.

`outputs = model(image)` — forward pass bajaradi.

`torch.softmax(outputs, dim=1)` — class scorelarni probabilityga aylantiradi.

`argmax(dim=1)` — eng katta probabilityga ega class indexni tanlaydi.

`.item()` — PyTorch tensor qiymatini oddiy Python soniga aylantiradi.

Mini xulosa sifatida deployment uchun training loop kerak emas.
Asosiy narsa quyidagi to'rtlik:

```
load model  
preprocess  
inference  
serve result
```

Serve result — natijani foydalanuvchiga qaytarish.

Deploymentda ishlatiladigan tools:

```
FastAPI  
Gradio  
Streamlit  
ONNX Runtime  
Docker  
cloud/server deployment
```

Docker — projectni bir xil environmentda ishga tushirish uchun container texnologiyasi. Container — kod, kutubxonalar va sozlamalar birga joylashgan paket.

Cloud/server deployment — modelni internetdagi serverga joylash.

Bu flow beginner projectlar uchun juda qulay. Avval model train qilinadi, keyin model fayli saqlanadi, undan keyin Gradio yoki FastAPI orqali oddiy inference app qilinadi.

Deployment and Deployment Optimization

Deployment va Deployment Optimization bir-biriga yaqin, lekin vazifasi farq qiladi. Deployment modelni foydalanuvchi ishlata

oladigan tizimga olib chiqadi. Deployment Optimization esa modelni shu tizimda tezroq, kichikroq va samaraliroq ishlaydigan qiladi.

Deployment simple view misolida trained model app yoki serverga deploy qilinadi. Trained model ready to deploy holatda bo‘ladi. Model app on device yoki server cloud/on-prem muhitga joylanadi.

On device — model qurilmaning o‘zida ishlashi. Masalan, telefon ichida.

Cloud — internetdagi server muhiti.

On-prem — kompaniya yoki tashkilotning o‘z serverida ishlatish.

Bu simple viewda maqsad modelni foydalanuvchilar uchun mavjud qilish hisoblanadi. “Make the model available to users” degan mazmun shuni bildiradi: model oddiy fayl bo‘lib qolmaydi, undan foydalanuvchi foydalanishi mumkin.

Deployment technical viewda model file API/server orqali foydalanuvchiga xizmat qiladi. Model file .onnx bo‘lishi mumkin. API/server modelni host qiladi. Host — modelni serverda ishga tushirib turish. User/client request yuboradi. Server response qaytaradi.

Oqim:

Model File (.onnx)

↓

API / Server hosts the model

↓

User / Client sends request

↓

Response output qaytadi

Request — foydalanuvchi yuborgan so‘rov.

Response — server qaytargan javob.

Deployment Optimization simple viewda original model katta va sekin bo‘ladi. Optimizationdan keyin model kichikroq va tezroq bo‘ladi.

Original Model → Optimize → Optimized Model
Large, slower → Smaller, faster

Original Model — boshlang‘ich model.

Optimized Model — optimallashtirilgan model.

Deployment Optimization technical viewda jarayon quyidagicha:

Trained Model

↓

Quantization

↓

Export to ONNX

↓

TensorRT Optimization

↓

Runtime Optimization

↓

Optimized Model

Trained Model high precision bo‘lishi mumkin. High precision — yuqori aniqlikdagi son formatlari bilan ishlash. Masalan, float32.

Quantization int8 kabi formatga o‘tkazadi. Bu modelni kichraytiradi.

Export to ONNX modelni interoperable qiladi. Interoperable — turli tizimlar orasida ishlay oladigan.

TensorRT Optimization engine building bosqichi bo‘lishi mumkin. Engine building — modelni aniq qurilmada tez ishlashga mos bajarilish formatiga keltirish.

Runtime Optimization execution paytida modelni tez ishlatishga yordam beradi.

Optimized Model quyidagi foydalarni beradi:

Lower Latency

Higher Throughput

Smaller Size

Lower Latency — kechikish kamroq.

Higher Throughput — ko‘proq so‘rovni qayta ishlash imkoniyati.

Smaller Size — kichikroq model hajmi.

Deployment Optimization product uchun juda muhim. Model local notebookda yaxshi ishlashi mumkin, lekin serverda sekin bo‘lsa yoki edge deviceda xotira yetmasa, foydalanuvchi uchun yaroqsiz bo‘lib qoladi.

Qayerga deploy qilish mumkin?

Modelni deploy qilish uchun ikki asosiy yo‘l bor:

Local / Edge Deployment

Cloud / Server Deployment

Local / Edge Deployment

Local / Edge Deployment — modelni lokal kompyuter, local server yoki kichik qurilmada ishlatish. Edge device — ma'lumot kelayotgan joyga yaqin qurilma. Masalan, Raspberry Pi, Jetson, laptop, kamera yoki offline app.

Local / Edge Deployment joylari:

Laptop

Local server

Raspberry Pi

Jetson

Offline app

Laptop — shaxsiy kompyuter.

Local server — lokal tarmoq ichidagi server.

Raspberry Pi — kichik kompyuter.

Jetson — NVIDIA tomonidan ishlab chiqilgan AI edge qurilma.

Offline app — internetsiz ishlaydigan ilova.

Local / Edge Deployment afzalliklari:

tez prototyping

maxfiylik yaxshi

internet shart emas

Prototyping — projectning tez sinov versiyasini yaratish. Beginner projectlarda avval local deployment qilish qulay, chunki hamma narsa kompyuterda tez sinab ko'riladi.

Maxfiylik yaxshi bo'lishi mumkin, chunki rasm internetdagi serverga yuborilmaydi. Masalan, tibbiy rasm yoki shaxsiy kamera tasviri lokal qurilmada qayta ishlansa, privacy kuchliroq bo'ladi.

Privacy — shaxsiy ma'lumotlar maxfiyligi.

Internet shart emas. Offline app internet bo'lmagan joyda ham ishlashi mumkin.

Local / Edge Deployment kamchiliklari:

scale cheklangan
qurilma kuchi muhim

Scale cheklangan — bitta laptop yoki kichik qurilma ko'p foydalanuvchini ko'tara olmasligi mumkin.

Qurilma kuchi muhim — model katta bo'lsa, Raspberry Pi yoki oddiy telefon uni tez ishlata olmasligi mumkin.

Cloud / Server Deployment

Cloud / Server Deployment — modelni internetdagi server yoki kuchli server muhitiga joylashtirish. Bu ko'p foydalanuvchi ishlatadigan productlar uchun qulay.

Cloud / Server Deployment joylari:

VPS

AWS

GCP

Azure

Render

Railway

On-prem server

VPS — Virtual Private Server. Internetdagi virtual server.

AWS — Amazon Web Services.

GCP — Google Cloud Platform.

Azure — Microsoft cloud platformasi.

Render va Railway — web app va API deploy qilish uchun qulay cloud platformalar.

On-prem server — tashkilotning oʻzida joylashgan server.

Cloud / Server Deployment afzalliklari:

koʻp foydalanuvchi uchun mos
API qilish oson
monitoring va scaling qulay

Koʻp foydalanuvchi uchun mos — server resursini oshirish orqali koʻproq requestni qabul qilish mumkin.

API qilish oson — backend server orqali foydalanuvchi requestlarini qabul qilib, modelga yuborish mumkin.

Monitoring va scaling qulay — cloud platformalarda logs, metrics, alerts va autoscaling kabi imkoniyatlar boʻladi.

Autoscaling — foydalanuvchi koʻpayganda server resurslarini avtomatik oshirish.

Cloud / Server Deployment kamchiliklari:

xarajat bor
internetga bog'liq

Xarajat bor — cloud server, GPU server, storage va traffic pul talab qilishi mumkin.

Internetga bog'liq — foydalanuvchi va server orasida internet bo'lishi kerak. Internet sekin bo'lsa, latency oshishi mumkin.

Boshlang'ich o'rganish uchun local deployment + Gradio yoki FastAPI qulay. Chunki murakkab cloud setup kerak bo'lmaydi. Avval local app ishlatiladi, keyin kerak bo'lsa cloudga chiqariladi.

Deploymentdagi muammolar va yechimlar

Deployment jarayonida model trainingda yaxshi ishlagan bo'lsa ham, real tizimga chiqqanda turli muammolar paydo bo'lishi mumkin. Bu muammolar model sifati, tezlik, xotira, environment, API va user experience bilan bog'liq bo'ladi.

Model sekin ishlayapti

Model sekin ishlasa, foydalanuvchi natijani uzoq kutadi. Bu latency oshganini bildiradi.

Yechimlar:

Quantization
ONNX
Smaller model
Batch tuning

Quantization modelni kichraytirib, inference tezligini oshirishi mumkin.

ONNX Runtime PyTorch modelga qaraganda ayrim holatda tezroq inference berishi mumkin.

Smaller model — kichikroq model ishlatish. Masalan, ResNet50 oʻrniga MobileNet ishlatish.

Batch tuning — batch size'ni moslash. Serverda bir nechta requestni bir batch qilib qayta ishlash throughputni oshirishi mumkin.

Memory yetmayapti

Memory — xotira. Model yoki input image juda katta boʻlsa, server yoki qurilmada xotira yetmasligi mumkin.

Yechimlar:

image size ni kichraytirish
lighter backbone ishlatish
modelni quantization qilish
batch size ni kamaytirish

Image size kichraytirilsa, model kamroq hisoblash qiladi. Masalan, 512×512 oʻrniga 224×224 ishlatish mumkin.

Lighter backbone — yengilroq backbone. Masalan, EfficientNet-B0 yoki MobileNet.

Prediction sifati pasaydi

Prediction sifati pasaysa, model real hayotdagi data'da training paytidagidek ishlamayotgan boʻlishi mumkin.

Yechimlar:

monitoring
retraining

data drift analysis
yangi data yig'ish
xato predictionlarni ko'rib chiqish

Retraining — modelni yangi data bilan qayta train qilish.

Data drift analysis — real data training data'dan qanchalik farq qilayotganini tahlil qilish.

Agar foydalanuvchilar trainingda bo'lmagan turdagi rasmlar yuborayotgan bo'lsa, model xato qilishi mumkin. Shu rasmlar yig'ilib, keyingi trainingda ishlatiladi.

Different environment error

Different environment error — model bir kompyuterda ishlaydi, boshqa serverda xato beradi. Bu kutubxona versiyalari, Python versiyasi, CUDA, PyTorch yoki path muammolari sababli bo'lishi mumkin.

Yechimlar:

requirements.txt
Docker
fixed versions
environment documentation

requirements.txt — Python kutubxonalari va versiyalari yoziladigan fayl.

Fixed versions — kutubxonalar versiyasini aniq belgilash. Masalan:

```
torch==2.3.0  
fastapi==0.111.0
```

Docker environmentni bir xil saqlashga yordam beradi. Docker container ichida kod ham, kutubxonalar ham birga bo‘ladi.

API timeout

API timeout — server belgilangan vaqt ichida javob qaytara olmasligi. Model sekin ishlasa, rasm juda katta bo‘lsa yoki server band bo‘lsa, timeout bo‘lishi mumkin.

Yechimlar:

- asynchronous queue
- caching
- stronger server
- model optimization

Asynchronous queue — requestlarni navbatga qo‘yib, fon rejimida qayta ishlash. Asynchronous — bir vaqtning o‘zida kutmasdan ishlash usuli.

Caching — oldin hisoblangan natijani vaqtincha saqlab, qayta ishlatish. Agar bir xil rasm yoki bir xil request qayta kelsa, modelni qayta ishlatmasdan oldingi natija qaytarilishi mumkin.

Stronger server — kuchliroq server yoki GPU ishlatish.

User experience yomon

User experience yomon bo‘lsa, model yaxshi ishlasa ham foydalanuvchi productdan foydalanishni xohlamasligi mumkin.

Yechimlar:

simple UI
progress indicator
clear output
error message
preview image
confidence ko'rsatish

Simple UI — oddiy va tushunarli interfeys.

Progress indicator — jarayon ketayotganini ko'rsatadigan belgi.
Masalan, loading spinner.

Clear output — natijani aniq ko'rsatish. Masalan:

Prediction: Dog
Confidence: 98%

Error message — xato bo'lsa, foydalanuvchiga tushunarli xabar berish. Masalan, “Faqat JPG yoki PNG rasm yuklash mumkin”.

Deploymentdagi muammolar va yechimlar jadvali shuni ko'rsatadi:
productga chiqarishda faqat model accuracy emas, tezlik, xotira, environment, API barqarorligi va foydalanuvchi interfeysi ham muhim.

Yakuniy umumiy tushuncha

Deployment & Inference Computer Vision projectining real hayotga chiqadigan qismi hisoblanadi. Model trainingdan o'tib yaxshi natija bergani bilan, foydalanuvchi undan foydalana olmasa, u amaliy product bo'la olmaydi. Deployment modelni foydalanuvchi ishlata oladigan tizimga joylashtiradi. Inference esa yangi rasm kelganda modelning prediction berishini ta'minlaydi.

Deployment Optimization modelni production uchun yengillashtiradi. Quantization model hajmini kamaytiradi. ONNX modelni turli runtime va frameworklarda ishlatishga yordam beradi. TensorRT NVIDIA GPUda inference'ni tezlashtiradi. Bu usullar model size, inference speed, latency va portabilityni yaxshilaydi.

Inference'da training loop ishlamaydi. Weightlar o'zgarmaydi. Model faqat forward pass qiladi. Rasm preprocess qilinadi, modelga beriladi, softmax orqali probabilities olinadi va eng katta probabilityga ega class predicted class sifatida tanlanadi.

Monitoring deploy qilingan modelning sog'lom ishlashini kuzatadi. Accuracy, latency, drift va failures kabi ko'rsatkichlar product sifatini nazorat qilishga yordam beradi. Latency 120 ms kabi qiymatlar model qanchalik tez javob berishini ko'rsatadi. Accuracy 98.2% modelning prediction sifati yaxshi ekanini bildiradi. Data drift va prediction errors esa model vaqt o'tishi bilan yomonlashayotgan yoki yomonlashmayotganini aniqlashga yordam beradi.

Cat vs Dog classifier misolida end-to-end oqim aniq ko'rinadi: foydalanuvchi rasm yuklaydi, app preprocess qiladi, model inference bajaradi, output sifatida Dog 0.98 qaytadi, log va monitoring tizimi jarayonni yozib boradi. Frontend foydalanuvchi interfeysini beradi, backend requestni qabul qiladi, model runtime prediction qiladi, monitoring esa response time va prediction distributionni kuzatadi.

Deployment local/edge yoki cloud/server muhitda qilinishi mumkin. Local deployment tez prototyping, privacy va offline ishlash uchun qulay. Cloud deployment esa ko'p foydalanuvchi, API, monitoring va scaling uchun qulay. Har bir variantning afzalligi va kamchiligi bor.

Deploymentdagi umumiy muammolar model sekin ishlashi, memory yetmasligi, prediction sifati pasayishi, environment error, API timeout va user experience yomon bo'lishi bilan bog'liq. Har bir

muammoning yechimi bor: quantization, ONNX, smaller model, image size kamaytirish, retraining, Docker, requirements.txt, asynchronous queue, caching, simple UI va clear output.

To'liq jarayon quyidagicha umumlashtiriladi:

Train qilingan model
↓
Deployment optimization
↓
API yoki app ichiga joylash
↓
Yangi rasmni qabul qilish
↓
Preprocessing
↓
Inference
↓
Predicted class + probability
↓
Monitoring
↓
Kerak bo'lsa retraining

Bu bosqich modelni notebook ichidagi tajribadan haqiqiy product holatiga olib chiqadi. Computer Vision projectining yakuniy foydasi aynan deployment, inference va monitoring orqali ko'rinadi.